

velocoach-rs Explained: From One Picture to Every Part

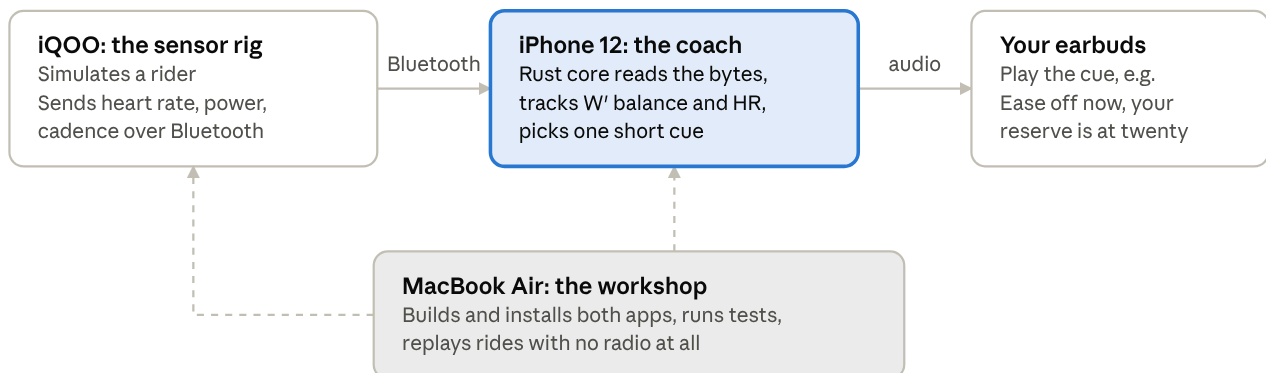
Oct 4, 2026 · @srivedya

1. The one-minute version

You are building a small, working copy of Domesly's core loop. A Rust engine listens to cycling sensors over Bluetooth, works out how hard the rider is pushing and how much reserve is left, and speaks one short coaching sentence at the right moment. You have no real sensors, so your iQOO phone pretends to be them.

A fake rider on one phone, a Rust coach on the other, a cue in your ear

Everything travels over real Bluetooth radio, exactly as a real sensor would send it



The whole project · 3 devices, 1 Rust brain

Read the picture left to right. The iQOO plays a virtual rider and sends the same bytes a real heart-rate strap and power meter would. The iPhone runs the Rust brain, decides what to say, and plays it into your earbuds. The Mac is where you build, test and replay everything.

In one line each:

- **What goes in:** heart rate, power and cadence, as raw Bluetooth bytes, 1 to 4 times a second.
- **What happens in the middle:** one Rust core turns bytes into numbers, numbers into a judgement ("reserve is low"), and the judgement into at most one cue.

- **What comes out:** a pre-recorded voice line of 8 to 12 syllables, verb first, at most one number.
- **Why it matters:** this is the exact job in Domesly's Rust role (BLE, real-time maths, mobile bridging, audio), built end to end and measured.

What you are not building: a full product, machine learning, an App Store release, Garmin or Strava sync, or any hardware. Those stay as one-line "next steps" in your write-up.

2. Objectives and what "done" means

The project has one purpose: make Aki think "he can own our Rust engine from day one". That breaks into four objectives, each with a number you can check.

1. **It works end to end on real devices.** A simulated ride on the iQOO produces spoken cues on the iPhone over real Bluetooth.
2. **It is fast, and you can prove it.** Sensor-to-decision latency is measured, not claimed.
3. **It never crashes on bad data.** Garbage packets, dropouts and disconnects are handled and tested.
4. **It follows Aki's own rules.** Safety cues bypass everything, and every spoken cue is verb first, has at most one number, and is 8 to 12 syllables.

#	Success criterion	Target	How you check it
1	Decision time inside Rust (bytes in, cue out)	under 1 ms at p99 on the iPhone 12	Timestamps T1 to T2
2	Sensor send to audio submitted	under 150 ms at p99, with the clock-sync error shown	Timestamps T0 to T3
3	Sensor send to your ear	reported honestly, not promised	T3 plus the iPhone's reported output latency
4	Bad bytes	zero panics after 10 minutes of fuzzing per parser	cargo fuzz
5	Repeatability	same scenario + same seed = identical cue log, every time	Golden-file tests
6	Endurance	60-minute run, no crashes, every disconnect recovered	Soak test

#	Success criterion	Target	How you check it
7	Phrase rules	100% of cues pass the verb / number / syllable test	<code>cargo test</code>
8	Embedded-ready maths	parsers and physiology build with <code>no_std</code> for a microcontroller target	CI build for <code>thumbv7em-none-eabi</code>

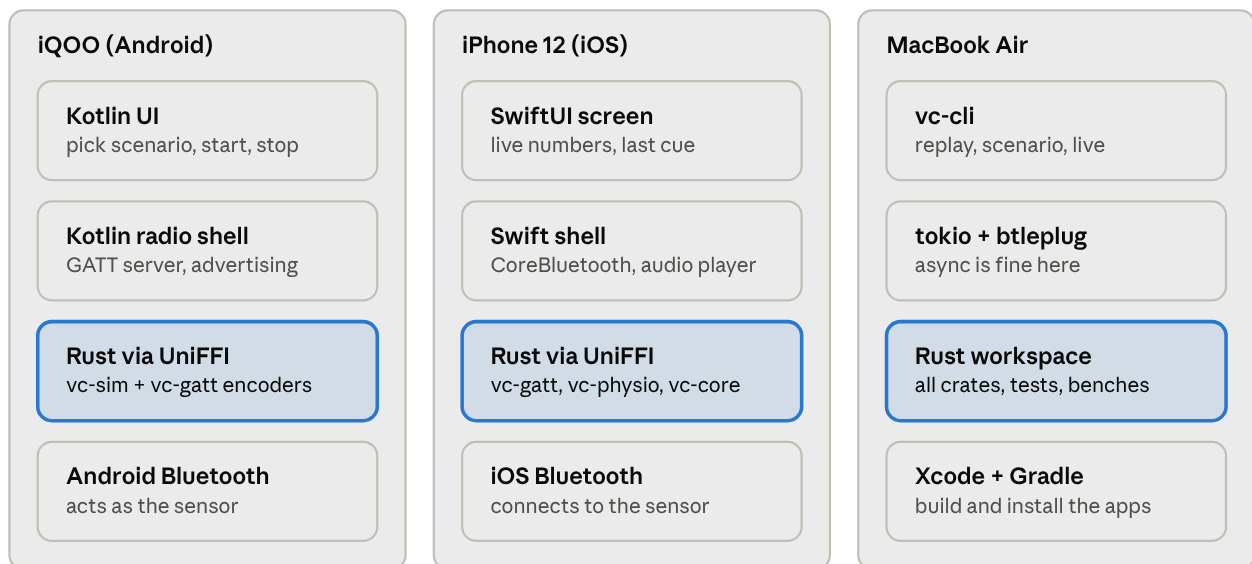
Criterion 3 is deliberately "report, don't promise": Bluetooth earbuds add their own delay (often over 100 ms) that no code can remove. Saying so plainly is part of what makes the work look senior.

3. The three devices and their jobs

Each phone runs a thin native shell around the same Rust code. The shell only does what the operating system forces it to do (Bluetooth, screen, sound). Every decision lives in Rust.

Thin native shells on each device, the same Rust crates underneath

Read each column top to bottom: screen, platform code, Rust, operating system



☒ Highlighted: the same Rust code, compiled for each device

What runs on each device · 4 layers per device

Device	Bluetooth role	Its one job	Why this device
iQOO	Peripheral (pretends to be the sensor)	Advertise heart-rate, power and cadence services and send packets from the simulated rider	Android apps can host standard sensor services reliably; iOS restricts some and drops service IDs when backgrounded
iPhone 12	Central (connects to the sensor)	Receive packets, run the Rust brain, play cues	Domesly is hiring a Swift developer, so their app is likely iOS-first
MacBook Air	Central, only when the iPhone is not connected	Build everything, run tests, replay rides offline, act as a second listener for debugging	It is your only machine that can build for both phones

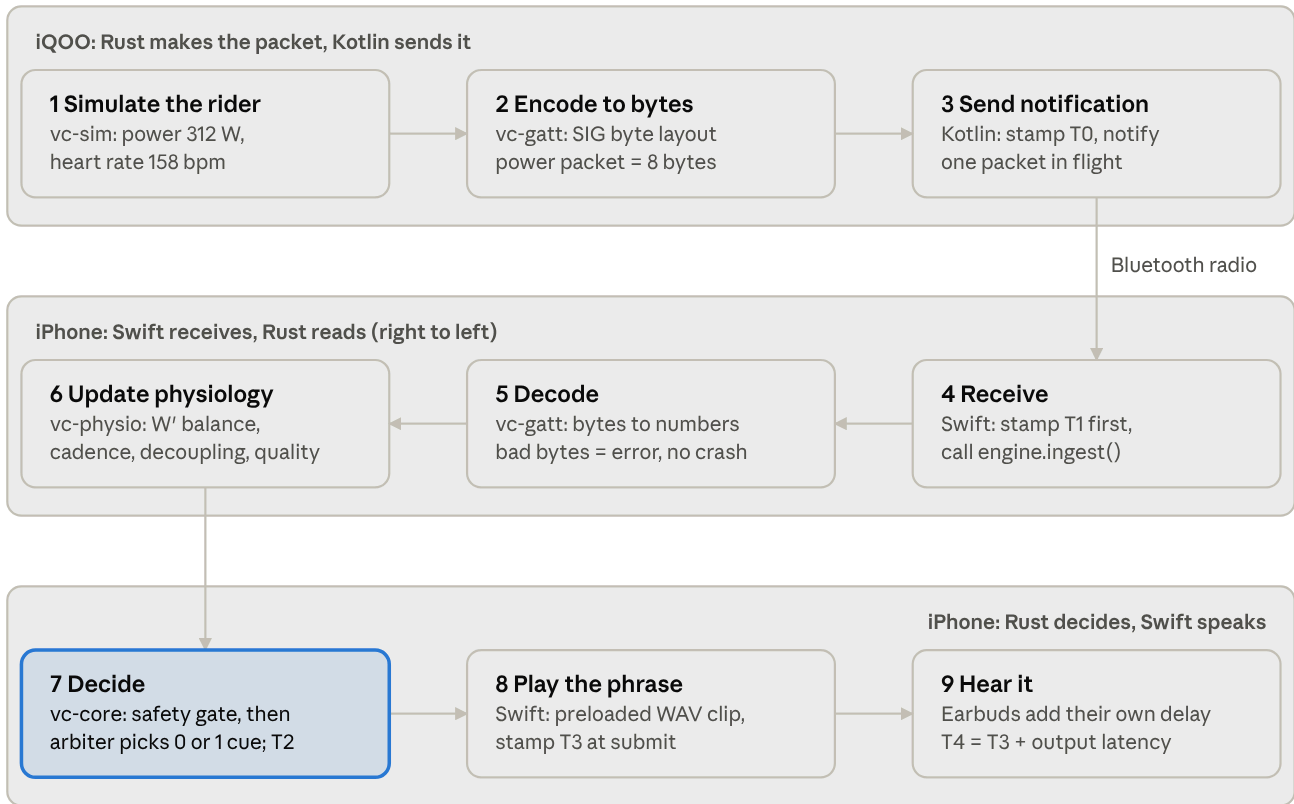
Two rules to remember. First, only one phone or Mac listens to the iQOO at a time. Second, nothing on the phones uses `tokio`: the operating systems already deliver Bluetooth events, so the Rust core stays a plain, synchronous library.

4. The journey of one reading

Follow a single power reading from the moment the fake rider produces it to the moment you hear a cue. If you understand this picture, you understand the whole system.

One reading travels nine steps from the fake rider to your ear

Steps 4 to 7 happen inside a single Rust function call on the iPhone



The life of one power reading · 9 steps, 5 timestamps

The five timestamps are how you prove the system is fast. Each one is stamped at a precise point:

Stamp	Where	Clock	Meaning
T0	iQOO, just before <code>notify</code>	iQOO's clock	The "sensor" sent it
T1	iPhone, first line of the Bluetooth callback	iPhone's clock	The coach received it
T2	iPhone, when Rust returns a cue	iPhone's clock	The decision was made
T3	iPhone, when the audio clip is scheduled	iPhone's clock	Sound was handed to iOS
T4	Estimated: T3 + iOS's reported output latency	iPhone's clock	Sound reaches your ear

T0 lives on a different phone, so comparing it with T1 needs a small clock-sync step (section 12).

Two ways a cue can be triggered:

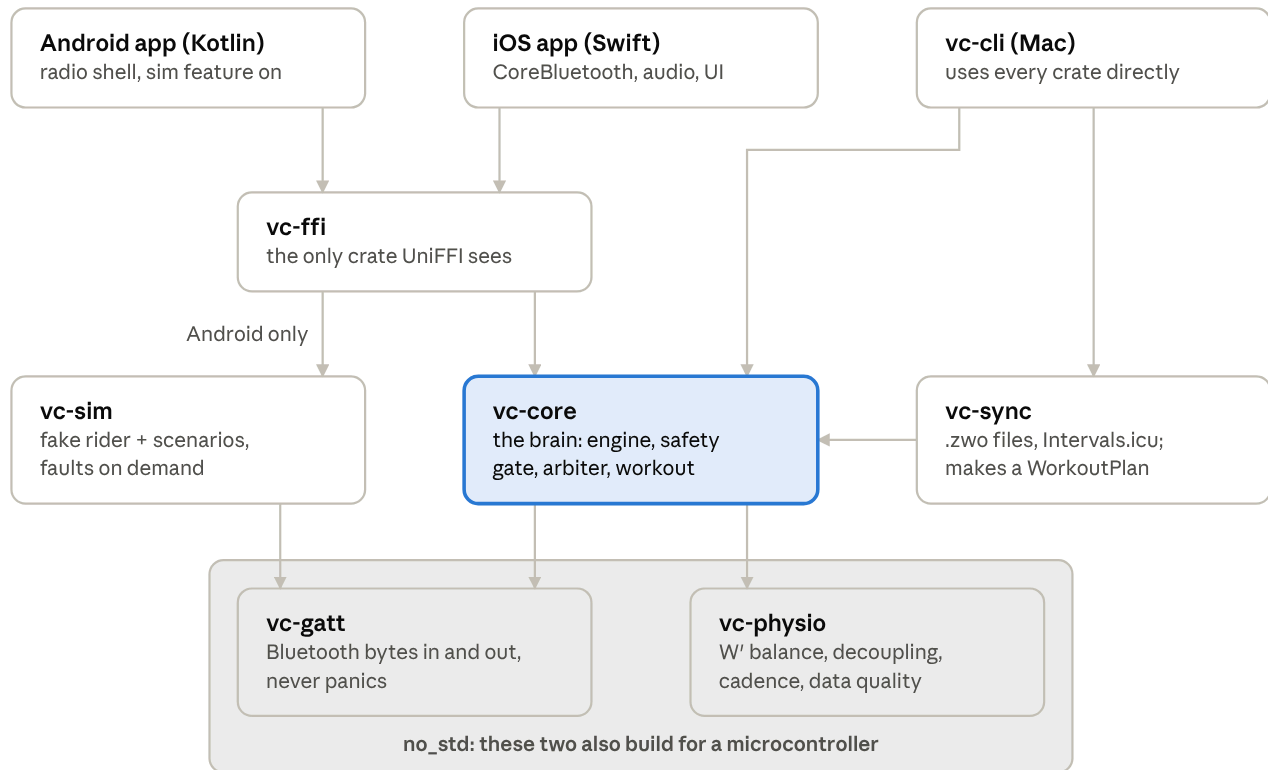
- **By data (event-driven):** every incoming packet runs steps 5 to 7 at once. This is how "your reserve is low" fires within milliseconds.
- **By time (a 4 Hz tick):** four times a second Swift calls `tick()`. This handles things no packet triggers: interval countdowns, a sensor going silent, and cues waiting for a quiet gap.

5. The code map

The project is one Cargo workspace with six crates, two phone apps and two small tools. The rule that makes it clean: logic flows downward, and only `vc-ffi` knows that phones exist.

Apps sit on one bridge crate; the logic lives in pure Rust crates

Arrows mean uses. Nothing below the bridge knows about phones.



The code map · 6 crates, 3 consumers

Crate	What it does in one line	Runs on	Allocates?
<code>vc-gatt</code>	Turns Bluetooth bytes into typed structs and back	iQOO, iPhone, Mac, microcontroller	No (<code>no_std</code>)
<code>vc-physio</code>	The sports-science maths: W' balance, decoupling, cadence, signal quality	iPhone, Mac, microcontroller	No (<code>no_std</code>)
<code>vc-sim</code>	A virtual rider whose heart, legs and fatigue behave believably, plus scripted faults	iQOO, Mac	Only at setup
<code>vc-core</code>	The brain: takes readings, runs the safety gate and arbiter, emits at most one cue	iPhone, Mac	Only at setup

Crate	What it does in one line	Runs on	Allocates?
<code>vc-sync</code>	Reads planned workouts (.zwo files, Intervals.icu) into a <code>WorkoutPlan</code>	Mac (iPhone later)	Yes, off the hot path
<code>vc-ffi</code>	Wraps <code>vc-core</code> and <code>vc-sim</code> as objects Swift and Kotlin can call	iQOO, iPhone	Small copies only

The folder layout you will create:

```

velocoach-rs/
├─ crates/
│   ├─ vc-gatt/      vc-physio/    vc-sim/
│   ├─ vc-core/      vc-sync/      vc-ffi/
├─ tools/
│   ├─ vc-cli/        (Mac harness)
│   └─ phrasegen/     (builds the voice clips)
├─ apps/
│   ├─ android/       (Kotlin sensor rig)
│   └─ ios/           (SwiftUI coach)
├─ scenarios/         (scripted rides, .toml + .zwo)
├─ scripts/           (build-ios.sh, build-android.sh)
└─ docs/              (README, LATENCY, ENGINEERING_NOTES, DEVLOG)

```

6. Deep dive: vc-gatt, the bytes on the wire

`vc-gatt` is a translator. Bluetooth sensors don't send "312 watts"; they send a few bytes in a layout fixed by the Bluetooth SIG. This crate turns bytes into typed Rust structs (decode) and structs back into bytes (encode). The simulator encodes; the coach decodes. Both use the same code, so they can never disagree.

A power packet is 8 bytes; the flags say which optional fields follow

Cycling Power Measurement (0x2A63), little-endian: the low byte comes first

	byte 0	byte 1	byte 2	byte 3	byte 4	byte 5	byte 6	byte 7
hex	20	00	38	01	D2	04	88	13
	flags = 0x0020 bit 5: crank data follows		power = 312 W signed, always present		crank revs = 1234 counter, wraps to 0		crank time = 5000 units of 1/1024 s	

One real power packet, byte by byte

How decoding works, step by step:

1. Read the flags field first. Each set bit means "an optional field is present".
2. Read the mandatory field (for power, the signed watts).
3. Walk the optional fields in bit order. For each set bit, read that field; skip unset ones.
4. If the bytes run out before a field is complete, return `Truncated`. Never index past the end, never panic.

A small `Cursor` helper does all the reading. Every read uses `.get()` and returns a `Result`, so a short or garbage packet becomes an error value, not a crash.

The three standard packets you handle:

Packet	ID	Size sent by the sim	What's inside
Heart Rate Measurement	0x2A37 (service 0x180D)	4 bytes	flags, heart rate (8 or 16 bit), optional energy, optional R-R intervals
Cycling Power Measurement	0x2A63 (service 0x1818)	8 bytes	flags (16 bit), watts (signed 16 bit), then optional fields by bit
Cycling Speed and Cadence	0x2A5B (service 0x1816)	5 bytes	flags (8 bit), optional wheel data, optional crank data

Traps that catch real apps (each one gets its own test):

- **Two time units in one family.** Wheel time in a power packet is in 1/2048 s; in a cadence packet it is 1/1024 s. Mixing them halves your speed.

- **Not every flag bit adds a field.** In the power packet, bits 1, 3 and 12 describe the data but add no bytes. Treating them as fields shifts every value after them.
- **Counters wrap.** Crank revolutions and event times are 16-bit. The time wraps every 64 seconds, so deltas must use `wrapping_sub`.
- **Coasting looks like repetition.** When the rider stops pedalling, the sensor repeats the same revolution count and time, then may go silent. A repeat is not new data.
- **Heart rate can be 8 or 16 bit.** Bit 0 of the flags decides which.

Your own service, VC-SIM. Besides the standard packets, the iQOO exposes one custom service with two channels:

Channel	Direction	Used for
TELEM	iQOO → iPhone	<code>STAMP</code> (the T0 send time of each sensor packet), <code>PONG</code> (clock-sync reply), <code>STATUS</code> (queue drops, errors)
CTRL	iPhone or Mac → iQOO	<code>PING</code> (clock sync), <code>LOAD / START / STOP</code> a scenario, <code>FAULT</code> (inject a dropout, garbage, disconnect, redline)

Every VC-SIM message starts with a version byte and an opcode byte, and fits in 20 bytes, the default Bluetooth payload size.

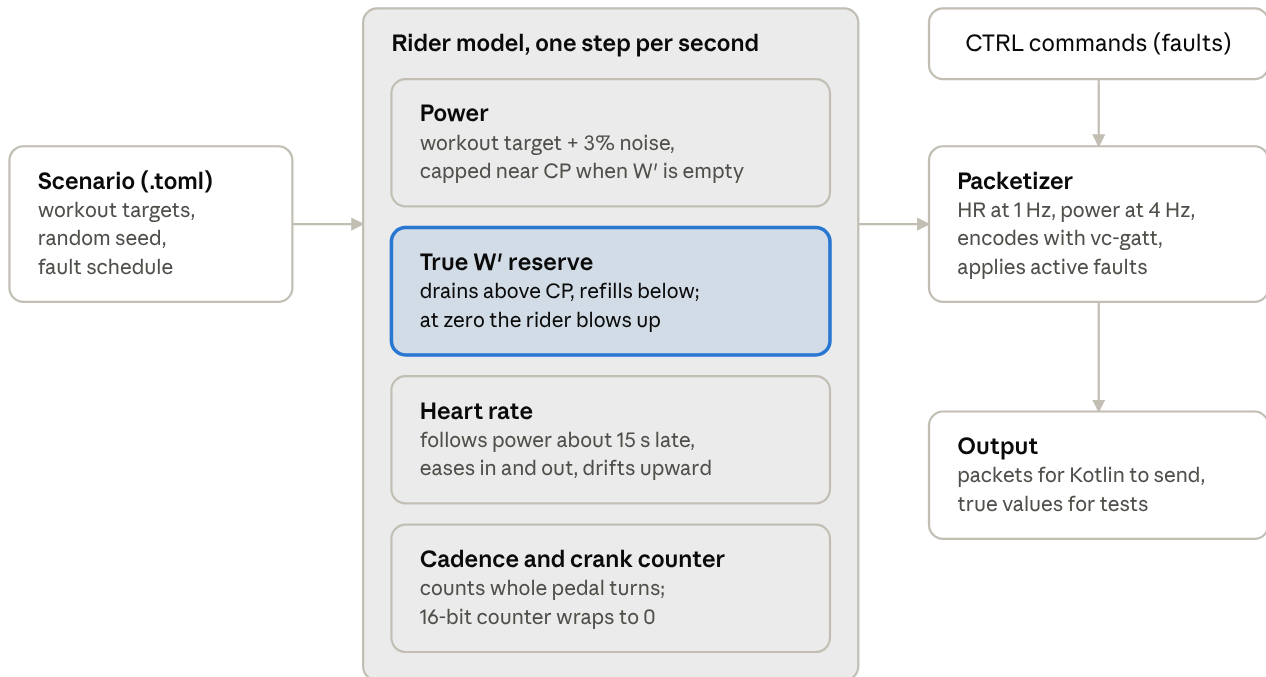
What you implement: `decode_hrm`, `decode_cpm`, `decode_csc`, `decode_telem`, `decode_ctrl` and a matching `encode_*` for each, all writing into a fixed 20-byte buffer with no heap allocation.

7. Deep dive: vc-sim, the fake rider

`vc-sim` replaces the bike, the legs and the heart. It must be believable enough that the coach's maths reacts as it would on a real ride, and controllable enough that you can make the rider fail on demand. It also records the "true" values, so tests can check that the coach decoded them correctly.

A scenario drives the rider model; the packetizer makes it Bluetooth

Same scenario + same seed = the same ride, byte for byte



Inside vc-sim · scenario in, packets and truth out

The four parts of the rider, in plain words:

- **Power.** Each second, the rider aims for the workout's target and misses by a little random noise (about 3%). If their true W' reserve is empty, they physically cannot hold the target: power drops to about 92% of CP until they recover.
- **True W' reserve.** The rider's own battery, using the same maths as section 8. It is deliberately separate from the coach's estimate, so you can test what happens when the coach's guess is wrong ("W' mismatch").
- **Heart rate.** Heart rate chases a target set by power, but about 15 seconds late. It rises with a time constant of about 35 s and falls more slowly (about 60 s). Over a long ride it creeps upward even at steady power; that is cardiac drift, and it is what decoupling measures.
- **Cadence.** The simulator counts whole pedal revolutions and stamps the time of the last one, exactly as a crank sensor does, including the 16-bit wrap.

The heart-rate rule as a formula (all constants are tunable defaults, not medical facts):

$$\frac{dHR}{dt} = \frac{HR_{\text{target}}(P(t - 15, s)) - HR}{\tau}, \quad \tau = 35, s \text{ rising}, \quad ; 60, s \text{ falling}$$

Scenarios you will script (each a `.toml` file plus a seed):

Scenario	What happens	What it proves
<code>vo2_5x3</code>	Warm-up, then hard intervals that drain W'	W' cues fire at the right moment
<code>blowup</code>	True W' is smaller than the coach thinks; the rider fades	The coach reacts to what it sees, not to the plan
<code>redline</code>	Heart rate climbs to max and stays there	The safety cue overrides everything
<code>dropout</code>	Heart-rate packets stop for 12 s	Stale and lost data are detected, cues are held back
<code>coast</code>	Peddalling stops; crank data repeats, then goes silent	No false "pedal faster" nagging
<code>garbage</code>	50 malformed packets	Nothing crashes; errors are counted

What you implement: `Simulator::new(scenario_toml, seed), step(now) -> Vec<SimPacket>` (called every 50 ms by Kotlin; returns any packets now due), `apply_ctrl(bytes)` for faults sent over the air, and `truth()` for tests. Randomness comes from a seeded generator (`rand_pcg`), never from the clock, so every run can be repeated exactly.

8. Deep dive: vc-physio, the maths

`vc-physio` turns raw readings into the four things a coach actually reasons about: how much anaerobic reserve is left (W' balance), whether the heart is drifting away from the power (decoupling), how fast the legs turn (cadence), and whether each sensor can be trusted right now (quality). It is pure maths with no allocation, so it also builds for a microcontroller.

W' balance: the anaerobic battery

Every rider has a critical power, **CP** (say 250 W), they can hold for a long time, and a fixed battery of extra energy, **W'** (say 20,000 J), for going above it. Ride above CP and the battery drains joule for joule. Ride below CP and it refills, faster the further below CP you are. Domesly's "protect your matches" cues come from this one number.

Above CP (draining), over a time step Δt :

$$W \rightarrow W - (P - CP) \Delta t$$

Below CP (refilling), the exact solution, which works even when packets arrive at uneven intervals:

$$W \rightarrow W' - (W' - W) e^{-(CP - P) \Delta t / W'}$$

This is the "differential" model GoldenCheetah uses by default. You also keep a second mode that copies GoldenCheetah's one-second steps exactly, so you can prove your numbers match theirs.

Test vectors (CP 250 W, W' 20,000 J). Write these as unit tests on day 4:

Start	Ride	Expected W' balance
Full (20,000 J)	350 W for 100 s	10,000 J
10,000 J	100 W for 133.33 s	about 16,321 J
Any value	exactly 250 W	unchanged
Empty (0 J)	0 W for 80 s	about 12,642 J

Decoupling: is the heart drifting?

At steady power, a well-trained rider's heart rate stays steady. When it creeps up, they are tiring. Decoupling compares the power-per-heartbeat ratio in the first half of a steady block against the second half:

$$\text{decoupling \%} = 100 \times \frac{EF_1 - EF_2}{EF_1}, \quad \text{quad } EF = \frac{\text{average power}}{\text{average heart rate}}$$

Rules: skip the first 10 minutes (warm-up), only use steady 30-second bins with good data, wait for at least 20 minutes of valid data, then fire an info cue at 5% or more and re-arm below 3.5%.

Cadence from crank counters

Sensors don't send cadence; they send a running count of pedal turns and the time of the last one. Cadence comes from the difference between two packets, using wrapping maths:

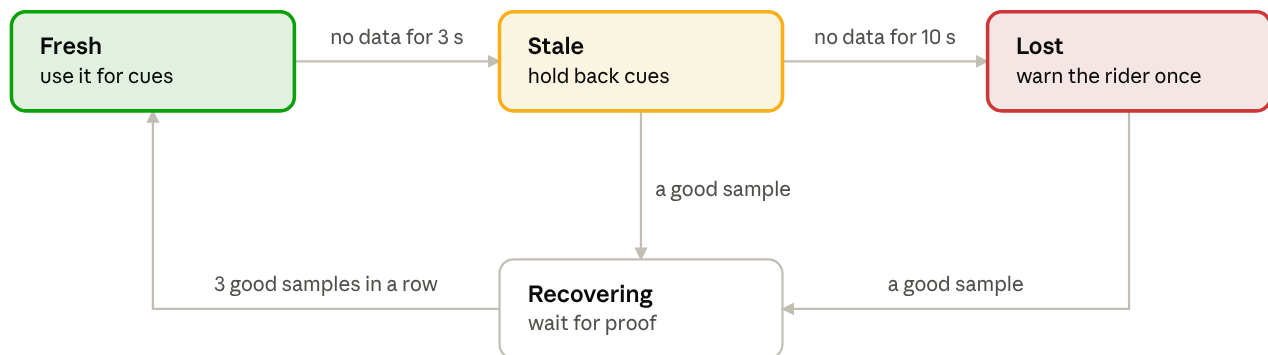
$$\text{rpm} = 60 \times \frac{\Delta \text{revs} \times 1024}{\Delta \text{time}}$$

If both differences are zero for more than 2.5 s, the rider is coasting: cadence is 0. Anything above 250 rpm is treated as garbage.

Data quality: can this sensor be trusted?

Each sensor channel walks through four states; only Fresh drives cues

Garbage packets never count as a good sample



Sensor data quality · 4 states, 5 transitions

Every channel (heart rate, power, cadence) carries one of these four states. The arbiter in section 9 refuses to give advice based on anything that is not Fresh, which is how the coach avoids telling you to "ease off" based on a number that is 20 seconds old.

What you implement: `WbalState::update(t_ns, watts)`, `Decoupler::push(...)`, `CadenceDeriver::push(revs, time, now)`, and `Quality::on_sample / Quality::on_tick`.

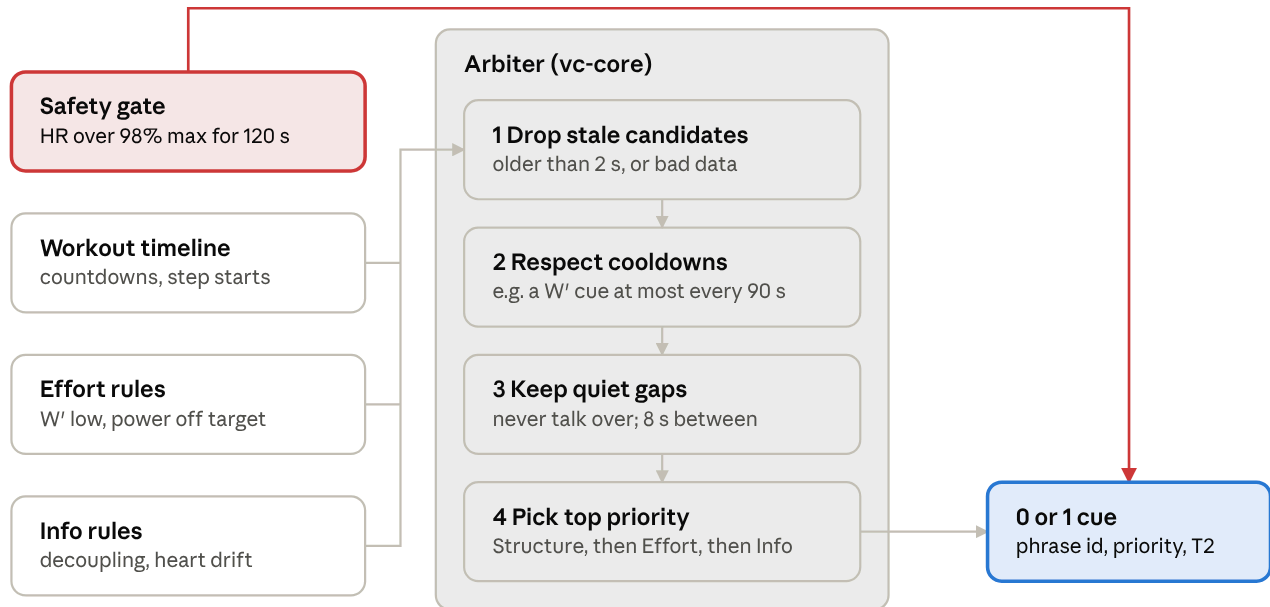
9. Deep dive: vc-core, the brain

vc-core holds an **Engine** that owns all the state for one ride. Every packet and every tick can produce cue candidates; the arbiter lets at most one through. This is the part that most directly reflects Aki's own design: a deterministic safety layer first, then disciplined, sparse coaching.

Every cue is a candidate; the arbiter lets at most one through

Four sources propose, four filters decide

safety skips the cooldown and gap checks, and may interrupt



How the brain decides · 4 sources, 4 filters, 1 bypass

The Engine's entry points

The platforms only ever call these. Everything else is internal.

Call	Who calls it, when	What it does
<code>ingest(t_ns, source, bytes)</code>	Swift, on every Bluetooth packet	Decode, update physiology and quality, run the gate and arbiter; may return a cue
<code>tick(t_ns)</code>	Swift, 4 times a second	Advance the workout clock, age the quality states, release a waiting cue

Call	Who calls it, when	What it does
<code>on_link(up, t_ns)</code>	Swift, on connect and disconnect	Mark channels, queue "sensor lost / back" cues
<code>on_audio_submitted(id, t3, latency)</code> and <code>on_audio_finished(id, t)</code>	Swift's audio player	Record T3/T4, end the "speaking" state
<code>snapshot()</code>	SwiftUI, 4 times a second	Copy out numbers for the screen
<code>latency_csv()</code>	The share button	Export the measurements

The safety gate

It runs before anything else, on every packet. Its rules are fixed code, never a model's opinion.

Rule	Fires when	Re-arms when
Heart-rate redline	Fresh HR above 98% of max for 120 s in a row	HR below 95% of max for 30 s
Sensor lost	A channel enters Lost	It returns to Fresh

The workout timeline

A `WorkoutPlan` is a list of steps (warm-up, interval, recovery, cool-down), each with a start, duration and power target. The timeline proposes a countdown 10 seconds before each boundary ("Get ready, interval in ten seconds") and a start cue at the boundary. Structure cues reserve the next 8 seconds, so an effort cue never steps on a countdown.

Effort rules and hysteresis

Hysteresis stops nagging. The W' cue fires when the reserve drops below 25%, then stays silent until it climbs back above 35%. Without that gap, a rider hovering at 24–26% would hear the same cue again and again.

Priorities and cooldowns

Priority	Examples	Cooldown	Special rights
Safety	redline, sensor lost	60 s	skips cooldowns and gaps, may interrupt
Structure	countdown, step start	none	reserves the next 8 s
Effort	W' low, power high or low, cadence	45–90 s per kind	none
Info	decoupling, drift	10 min	lowest; dropped first

What you implement: `Engine`, `SafetyGate`, `Timeline`, `Arbiter` with fixed-size slots (one per cue kind, no heap in the hot path), and tests that feed scripted inputs and assert exactly which cue comes out and when.

10. Deep dive: vc-ffi and the two apps

`vc-ffi` is the bridge. You write normal Rust, add a few UniFFI annotations, and a tool generates Swift and Kotlin code that calls it as if it were native. Pin `uniffi = "=0.32.2"` so the generated code never changes under you.

How the bridge is made, in four steps:

1. Annotate Rust types: `#[derive(uniffi::Object)]` on `Engine` and `Simulator`, `#[uniffi::export]` on their methods, `#[derive(uniffi::Record)]` on plain data like `CueOut` and `Snapshot`.
2. Build the library for each phone: a static library (`.a`) for iOS, a shared library (`.so`) for Android via `cargo-ndk`.
3. Run `uniffi-bindgen` on that library. It writes `vc_ffi.swift` (plus a header) and `vc_ffi.kt`.
4. Package for each app: an XCFramework inside a local Swift package for iOS; `jniLibs/arm64-v8a/` plus the JNA library for Android.

What crosses the bridge:

Rust item	Kind	Used by	Purpose
<code>Engine</code>	Object	iOS	The coach: <code>ingest</code> , <code>tick</code> , <code>snapshot</code> , audio callbacks
<code>Simulator</code>	Object (Android build only)	Android	The rider: <code>step</code> , <code>apply_ctrl</code>
<code>CueOut</code> , <code>Snapshot</code> , <code>SimPacket</code>	Records	both	Plain data copied across
<code>VcError</code>	Error enum	both	Becomes a Swift <code>throws</code> / Kotlin exception

One detail that bites in UniFFI 0.32: byte slices cross without copying. Swift simply passes `Data`. Kotlin must pass a **direct** `ByteBuffer` (`ByteBuffer.allocateDirect(n)`), not a normal byte array.

Threads: who may call Rust

Threads on each phone · who is allowed to call Rust

UniFFI objects must be safe to share between threads, so each one wraps its state in a `Mutex`. In practice each phone funnels every call through one serial thread, so the lock is never contended and timing stays predictable.

The iPhone app (Swift), piece by piece

File	Job
<code>BleCentral.swift</code>	Scan for the power service, connect, subscribe to all four channels, stamp T1, call <code>ingest</code> ; reconnect on drop
<code>AudioCuePlayer.swift</code>	Preload every phrase as an audio buffer; play a cue, stamp T3, ducking other audio
<code>RideView.swift</code>	The live screen: power, HR, cadence, W' %, current step, last cue, sensor states

File	Job
<code>Info.plist</code>	Bluetooth usage text, background modes <code>bluetooth-central</code> and <code>audio</code>

The iQOO app (Kotlin), piece by piece

File	Job
<code>SensorRigService.kt</code>	Foreground service of type <code>connectedDevice</code> , so Android doesn't kill it
<code>GattRig.kt</code>	Add the four services one at a time (wait for each confirmation), then advertise; track subscriptions
<code>RadioLoop.kt</code>	Every 50 ms ask Rust for due packets; send one notification, wait for "sent", then the next
<code>AndroidManifest.xml</code>	Permissions: advertise, connect, foreground service, notifications

The iQOO's battery saver is the biggest practical risk: keep the app on screen with the display on during runs, and set its battery usage to unrestricted.

11. Deep dive: the voice

The coach never "speaks" in real time. Every sentence is written, checked against Aki's rules, and recorded on the Mac before the ride. During the ride the iPhone only looks up a clip and plays it. That is why speaking adds almost no delay, and why Domesly's own website uses pre-recorded clips too.

The voice pipeline · build once, play many

The phrase rules, enforced by a test that fails the build:

1. The first word is a verb (from an allowed list: ease, hold, raise, start, check, drink...).
2. At most one number phrase ("two eighty" counts as one).
3. Between 8 and 12 syllables, counted from a word list you keep in the repo. An unknown word fails the test, so the count is never a guess.

A sample of the phrase bank (about 50 lines plus number words, roughly 70 clips):

Priority	Phrase	Syllables
Safety	Ease off now, your heart rate is too high	9
Safety	Check your sensor, heart rate signal lost	9
Structure	Get ready, interval in ten seconds	10
Structure	Begin recovery, spin easy now	10
Effort	Ease off, your reserve is at {twenty} percent	11
Effort	Raise your cadence, target {ninety}	8
Effort	Hold this power, you are on target	9
Info	Ease the pace, your heart rate is drifting	9

Words in braces are separate clips joined at play time. Numbers are snapped to a small fixed vocabulary (ten, twenty, thirty, ninety, two hundred, two eighty...) so every combination already exists as audio.

Playback rules on the iPhone:

- Load every clip into memory at app start (about 12 MB), so playing never touches the disk.
- Use the audio session category `.playback` with mode `.voicePrompt` and option `.duckOthers`: your music dips under the cue, then comes back.
- Safety cues interrupt whatever is playing. Every other cue waits for silence.
- A cue that is more than 2 seconds old when its turn comes is dropped, not spoken late.
- When the earbuds disconnect or a call comes in, the player pauses and the engine is told the cue never played.

12. Conversations between the devices

The two phones talk in three phases: a one-time setup, a clock sync that repeats, and a steady stream of sensor packets. Each sensor packet is followed by a tiny STAMP message carrying the iQOO's send time, which is what makes latency measurable.

The Bluetooth conversation · setup, clock sync, steady state

Clock sync, in plain words

The phones have unrelated clocks. To compare T0 (iQOO) with T1 (iPhone), the iPhone asks "what time is it?" and times the round trip. The classic NTP formulas give the offset between the clocks and an honest error bound:

$$\text{rtt} = (d - a) - (c - b), \quad \theta = \frac{(b - a) + (c - d)}{2}, \\ |\text{error}| \leq \frac{\text{rtt}_{\min}}{2}$$

Here a is when the iPhone sent PING, b when the iQOO received it, c when the iQOO sent PONG, and d when the iPhone received it. Keep the reply with the smallest round trip from the last 16: it is the least disturbed by radio delays. Every latency number you publish carries that $\pm$ error.

Reconnect after a drop

1. iOS reports the disconnect. The Engine is told the link is down; channels age to Stale (3 s), then Lost (10 s), and one "sensor lost" cue is queued.
2. The iPhone immediately asks to reconnect (iOS keeps that request open with no timeout) and also starts a fresh scan, in case Android changed its address.
3. On reconnect: rediscover, resubscribe, send 8 quick PINGs to resync clocks.
4. Channels move to Recovering, and back to Fresh after 3 good samples. A "sensors are back" cue plays.
5. On the iQOO side, the subscription list and the "packet in flight" flag are cleared, and advertising restarts if it stopped.

Injecting a fault on purpose

1. From the iPhone's debug sheet (or `vc-cli` on the Mac), write a CTRL `FAULT` message, for example "drop heart rate for 12 seconds".
2. Kotlin hands the bytes straight to Rust's `apply_ctrl`, which validates them and switches the fault on.
3. The simulator stops sending heart-rate packets for 12 s.
4. On the iPhone you watch HR go Stale, then Lost, then Recovering, then Fresh, with exactly the cues the golden test expects.

13. Measuring latency honestly

The JD says "sub-150 ms". You will show exactly which part of the chain that number covers, measure it, and report the part you cannot control separately. That honesty is the strongest signal in the whole project.

The latency timeline · 5 stamps, 4 segments

Segment	Includes	Clock	Target
T0 → T1	Android send queue, radio, iOS Bluetooth stack	Two phones, after clock sync (± error shown)	Measure and report
T1 → T2	Decode, physiology, safety gate, arbiter	iPhone only	Under 1 ms at p99
T2 → T3	Looking up and scheduling the clip	iPhone only	Under 20 ms at p99
T0 → T3	Everything you built	Synced	Under 150 ms at p99
T3 → T4	iOS output buffer plus Bluetooth earbud delay	Estimated from iOS's reported output latency	Report only

How you record it. Each segment gets its own histogram (`hdrhistogram` , values in nanoseconds), created once when the Engine starts so recording never allocates.

`latency_csv()` exports one row per cue: `cue_id`, `t0`, `t1`, `t2`, `t3`, `t4`, `sync_error`, `route` .

The measurement runs (day 13):

1. Three 20-minute runs of the interval scenario with your earbuds.
2. One run with the iPhone speaker as a control, so the earbud cost is visible as a difference.
3. Report p50, p90, p99 and max for every segment, plus the clock-sync error bound.

The table you will publish in `LATENCY.md` :

Segment	Route	Samples	p50 (ms)	p90 (ms)	p99 (ms)	max (ms)	Sync ± (ms)
T1 → T2	any						n/a
T0 → T3	earbuds						
T0 → T4	earbuds						
T0 → T4	speaker						

14. When things go wrong

A coach that crashes mid-ride, or confidently reads out a 30-second-old number, is worse than no coach. Every failure below has a planned detection, a planned reaction, and a test that proves it.

What fails	How you notice	What the system does	What the rider hears or sees	How you test it
Malformed packet	Decoder returns an error	Count it, ignore it, don't refresh the channel's quality	Nothing; a debug counter goes up	Fuzzing, garbage scenario
Sensor goes quiet	Quality state machine (Stale at 3 s, Lost at 10 s)	Hold back cues that depend on it	"Check your sensor..." once	dropout scenario
Bluetooth disconnect	iOS and Android connection callbacks	Reconnect and rescan, resync clocks, resubscribe	Status pill turns red, then green	Disconnect fault
Notification fails to send	Android returns a non-success code	Retry once after 20 ms, then drop and count	Nothing	Soak test
Bluetooth switched off	Adapter state callbacks	Pause; resume when it is back	A banner	Toggle Bluetooth by hand

What fails	How you notice	What the system does	What the rider hears or sees	How you test it
Permission denied	Authorisation check at start	Show why it is needed and a Settings link	A blocking screen	Deny it once by hand
iPhone remembers an old service list	A characteristic is missing after connect	Freeze the iQOO's service list; never change it while connected	An error message	Change the list once on purpose
iQOO kills the app in the background	STATUS messages stop for 5 s or more	Foreground service, battery unrestricted, screen on	Disconnect flow	60-minute run with the screen off
Earbuds removed or a call comes in	Audio session notifications	Stop the cue; tell the engine it never played; drop cues older than 2 s	The cue is skipped	Pull the earbuds out mid-cue

Three rules sit underneath all of this:

1. **No panics in the core.** Parsers return `Result`, no `unwrap` outside tests, enforced with `#![deny(clippy::unwrap_used)]`.
2. **Never advise on stale data.** Every cue candidate carries the freshness of the data that produced it.
3. **Degrade, don't die.** Losing power data still leaves heart-rate cues working, and losing both still leaves the workout countdowns running.

15. Proving it works

Tests are your evidence. A reviewer who opens the repo should be able to run one command and watch every claim in the README hold up.

The test pyramid · 5 layers

Layer	Tool	Example	What it proves
Unit	<code>cargo test</code>	A 4-byte heart-rate packet decodes to 158 bpm with one R-R value	Each function does its one job
Property	<code>proptest</code>	For any random power struct, <code>decode(encode(x)) == x</code>	Encoder and decoder agree on every input
Fuzz	<code>cargo fuzz</code>	Feed random bytes to every decoder for 10 minutes each	No input can crash the coach
Loopback	<code>cargo test</code>	Run <code>vc-sim</code> for 20 simulated minutes straight into <code>vc-core</code>	The whole pipeline works without phones
Golden logs	<code>insta</code> snapshots	The <code>redline</code> scenario with seed 42 always produces the same cue list	Behaviour never changes by accident
Hot path	counting allocator	10,000 <code>ingest()</code> calls after warm-up allocate nothing	Timing stays predictable
Speed	<code>criterion</code>	Benchmark <code>decode_cpm</code> and <code>WbalState::update</code>	Numbers for the README
Device	the two phones	60-minute run with a fault every 10 minutes	It survives the real world

Cross-checking W' against GoldenCheetah (the reference tool cycling coaches trust):

1. Export the simulator's true power as a 1-second CSV.
2. Import it into GoldenCheetah with CP 250 W and W' 20,000 J.
3. Compare GoldenCheetah's minimum W' balance with `vc-cli wbal --gc-parity`.
4. Accept a difference of 1.5% or less (GoldenCheetah smooths power slightly first).

CI on every push (GitHub Actions, free for public repos): format check, `clippy` with warnings as errors, all tests, a build of `vc-gatt` and `vc-physio` for the microcontroller target `thumbv7em-none-eabihf`, the golden-log tests, and a 60-second fuzz smoke run per decoder.

16. Build order: 20 days in five phases

The order is chosen so that something real works as early as possible. By day 11 the full loop runs on both phones; everything after that makes it measured, robust and presentable.

The 20-day roadmap · 5 phases, 3 release tags

Day by day (tick them off as you go):

- ☐ Day 1: Install tools. Use nRF Connect on the iQOO to fake a heart-rate sensor, and a tiny iPhone app that prints its bytes. Create the workspace and CI.
- ☐ Day 2: `vc-gatt` heart-rate and cadence decoders with byte-vector tests.
- ☐ Day 3: Power decoder, VC-SIM messages, round-trip property tests, fuzz targets, `no_std` build in CI.
- ☐ Day 4: W' balance (exact and GoldenCheetah modes), quality state machine, cadence deriver, with the test vectors from section 8.
- ☐ Day 5: Decoupling, simulator heart-rate model, drift and blow-up.
- ☐ Day 6: Crank counters, packet scheduler, scenario files, faults, loopback test.
- ☐ Day 7: Engine, workout timeline, safety gate, arbiter, `.zwo` parser.
- ☐ Day 8: Golden logs, phrase lint, `vc-cli`, `vc-ffi`, both build scripts produce artifacts.
- ☐ Day 9: Android sensor rig: permissions, foreground service, GATT server, advertising, radio loop. Check it from the Mac.
- ☐ Day 10: iPhone coach: connect, subscribe, ingest, tick, live screen, reconnect.
- ☐ Day 11: Render the phrase bank, play cues, run the full loop. **Tag v0.1.0.**
- ☐ Day 12: Clock sync, STAMP matching, latency histograms, CSV export.
- ☐ Day 13: Latency runs, fix the slowest segment, write `LATENCY.md`.
- ☐ Day 14: Walk every row of the failure table on the devices; re-sign the iPhone app (free profiles expire after 7 days).
- ☐ Day 15: Golden logs three times, 30-minute fuzz runs, allocation test, benchmarks. **Tag v0.2.0.**
- ☐ Day 16: Pull a workout from Intervals.icu and drive countdown cues from it.
- ☐ Day 17: Two 60-minute soak runs with a fault every 10 minutes.
- ☐ Day 18: GoldenCheetah cross-check, FIT/CSV replay, screen polish.
- ☐ Day 19: README, engineering notes, licences, a full dry run.

- ☐ Day 20: Record the 3-minute video, re-sign the app, final short soak. **Tag v1.0.0** and message Aki.

If you fall behind, cut in this order: FIT replay, then the second W' model, then benchmarks, then the Intervals.icu fetch (keep `.zwo` files), then the decoupling cue, then the GoldenCheetah cross-check.

Never cut: the safety gate, the arbiter tests, the golden logs, the latency measurement and the fuzzing. Those are the things that make the project look like professional work.

17. Glossary

Term	Meaning in this project
BLE	Bluetooth Low Energy, the radio that bike sensors and phones use
GATT	The way BLE organises data: services contain characteristics, which hold values
Service / characteristic	A group of related values ("Cycling Power") and one value inside it ("Power Measurement")
Peripheral / central	The device that offers data (the sensor, here the iQOO) / the device that connects to it (the iPhone)
Notification	A packet the peripheral pushes to the central whenever a value changes
CCCD	The switch (descriptor 0x2902) the central writes <code>01 00</code> to, to turn notifications on
SIG	Bluetooth Special Interest Group, which defines the standard byte layouts
Little-endian	Multi-byte numbers are sent low byte first: $312 = 0x0138$ travels as <code>38 01</code>
CP	Critical power: the highest power a rider can sustain for a long time
W' (W prime)	The fixed store of energy, in joules, a rider can spend above CP

Term	Meaning in this project
W' balance	How much of that store is left right now
Decoupling	How far the heart rate drifts upward relative to power during a steady effort
Cadence	Pedal revolutions per minute
FTP	Functional threshold power; a common training reference close to CP
Hysteresis	Using a lower trigger and a higher reset level so a cue doesn't repeat when a value hovers
Arbiter	The part of <code>vc-core</code> that chooses at most one cue at a time
Safety gate	Fixed rules that override everything else
Sans-IO	A library that never touches the network, radio or files; the caller feeds it data
<code>no_std</code>	Rust built without the standard library, so it can run on a microcontroller
UniFFI	Mozilla's tool that generates Swift and Kotlin bindings for Rust code
XCFramework	Apple's package format for a compiled library used by an iOS app
<code>cargo-ndk</code>	The tool that builds Rust for Android
Golden log	A saved, reviewed output; tests fail if a later run differs
Fuzzing	Throwing huge amounts of random input at a function to find crashes
p50 / p99	The value half of the measurements / 99% of them stay under
Soak test	A long run meant to surface slow failures such as leaks or disconnects
A2DP	The Bluetooth audio profile earbuds use; it adds noticeable output delay

Term	Meaning in this project
Piper	A free, offline text-to-speech engine used once on the Mac to record the phrases